

MODULE 5

Node.js Fundamentals

Backend Developer Documentation

12 Lessons • ~100 Minutes • Practical Code Examples

A complete reference guide covering the architecture, core modules, and essential patterns every Node.js backend developer must master.

Created By
Md Parvez Hasan

Table of Contents

- 5-1** How the Web Works from Scratch — 12 min
 - 5-2** Frontend vs Backend Essentials — 7 min
 - 5-3** Tracing Where Everything Began — 6 min
 - 5-4** How Node.js Revolutionized the Server Landscape — 4 min
 - 5-5** Event-Driven Architecture of Node.js — 11 min
 - 5-6** Single-Threaded Node.js vs Multi-Threaded Servers — 13 min
 - 5-7** How the Event Loop Works — 6 min
 - 5-8** Streams, Buffers & Modular Systems Deep Dive — 10 min
 - 5-9** Harnessing IIFE as the Ultimate Module Wrapper — 8 min
 - 5-10** Exploring CommonJS & Name Alias — 17 min
 - 5-11** Transforming to ESM — 5 min
 - APP** Core Modules Quick Reference (fs, http, path, os, events)
-

LESSON 5-1 • 12 min

How the Web Works from Scratch

Every backend developer must understand the full request-response lifecycle before writing a single line of server code. The web is built on a layered stack of protocols, each responsible for a specific concern.

The Full HTTP Request Journey

- 1. DNS Resolution:** Browser queries a DNS resolver to convert a domain name (e.g. example.com) into an IP address. Results are cached at browser, OS, router, and ISP levels.
- 2. TCP Handshake:** A reliable connection is established via the three-way handshake: SYN → SYN-ACK → ACK. This guarantees ordered, error-checked delivery.
- 3. TLS Handshake (HTTPS):** The client and server negotiate a cipher suite, exchange certificates, and derive session keys before any HTTP data is sent.
- 4. HTTP Request:** The browser sends an HTTP request with a method (GET/POST/...), headers (Host, Cookie, Content-Type), and optional body.
- 5. Server Processing:** Node.js receives the request, runs middleware and route handlers, queries databases or external services, then constructs a response.
- 6. HTTP Response:** The server replies with a status code (200, 404, 500...), response headers, and a body (HTML, JSON, binary...).
- 7. Rendering:** The browser parses HTML, fetches sub-resources, executes scripts, and paints the page.

Key Protocols at Each Layer

Layer	Protocol	Role
Application	HTTP/HTTPS, WebSocket	Data format & semantics
Transport	TCP, UDP	Reliability & port routing
Network	IP (IPv4/IPv6)	Logical addressing & routing
Data Link	Ethernet, Wi-Fi	Physical frame delivery

**Backend
Insight**

Node.js operates at the Application layer. The OS kernel handles TCP/IP, so your code only deals with HTTP semantics, headers, and bodies — a huge productivity win.

LESSON 5-2 • 7 min

Frontend vs Backend Essentials

Understanding the boundary between client and server is critical — it determines what code runs where, what data is exposed, and who is responsible for security.

Dimension	Frontend	Backend (Node.js)
Runs on	User's browser / device	Server / cloud
Language	JavaScript, HTML, CSS	JavaScript (Node.js)
Visibility	Fully visible to user	Hidden from client
State	UI state, local storage	Sessions, DB, cache
Security	Input validation (UX)	Auth, authorization, sanitization
I/O	DOM, fetch, WebSocket	File system, DB, network
Performance	Rendering, JS bundles	Throughput, latency, scaling

Node.js as the Full-Stack Bridge

Because Node.js uses JavaScript — the same language as the browser — teams can share validation logic, type definitions (via TypeScript), and utility functions across the entire stack. This is one of Node's most underappreciated advantages.

Golden Rule

Never trust the frontend. Always validate and sanitize ALL input on the server, regardless of what the client-side code does.

LESSON 5-3 • 6 min

Tracing Where Everything Began

Node.js didn't emerge in a vacuum. Understanding its history clarifies the design decisions that make it unique.

Year	Event
1991	Tim Berners-Lee publishes HTML & the first web server (written in C).
1994	Netscape Navigator launches; JavaScript created in 10 days by Brendan Eich.
1995–2008	PHP, ASP, Ruby, Python dominate. Each new request spawns a thread — the dominant model.
2008	Google releases the V8 JavaScript engine — blazing-fast JIT compilation of JS.
2009	Ryan Dahl creates Node.js, embedding V8 in a server runtime with non-blocking I/O.
2010	npm is released; the Node.js ecosystem explodes.
2015	Node.js Foundation formed; io.js fork reunites with Node core.
2020+	Node.js powers Netflix, LinkedIn, PayPal, Uber, NASA, and millions of services globally.

LESSON 5-4 • 4 min

How Node.js Revolutionized the Server Landscape

Before Node.js, the dominant server model was thread-per-request. Apache would spawn (or pool) OS threads for every incoming connection. This worked, but threads are expensive — each consumes ~1MB of stack memory and context-switching overhead.

The C10K Problem

In 1999, Dan Kegel posed the 'C10K problem': how do you handle 10,000 simultaneous connections? Thread-per-request servers collapsed under this load. The solution was non-blocking, event-driven I/O — which became Node's foundation.

What Node Changed

Single Process, Non-Blocking I/O: One Node process handles thousands of connections without spawning threads. I/O operations are offloaded to the OS and libuv.

JavaScript on the Server: Reuse JS skills, share code with the browser, unified tooling.

npm Ecosystem: Over 2 million packages. The largest software registry in the world.

Microservices & APIs: Node's lightweight footprint and fast startup time make it ideal for microservices, REST APIs, and real-time apps.

Key Fact

LinkedIn replaced its Rails backend with Node.js and reduced server count from 30 to 3 while handling 2x the traffic.

LESSON 5-5 • 11 min

Event-Driven Architecture of Node.js

Node.js is built on the event-driven programming paradigm. Instead of waiting for operations to complete (blocking), Node registers callbacks and immediately moves on. When the operation finishes, an event is emitted and the callback executes.

The EventEmitter Class

The core of Node's event system is the EventEmitter class from the events module. Every object that emits events in Node — HTTP servers, streams, processes — inherits from it.

```
// Basic EventEmitter pattern
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Register a listener
emitter.on('data', (payload) => {
```

```
console.log('Received:', payload);
});

// Emit an event
emitter.emit('data', { id: 1, value: 'hello' });
```

EventEmitter API Reference

Method	Description
<code>on(event, listener)</code>	Register persistent listener
<code>once(event, listener)</code>	Register one-time listener
<code>emit(event, ...args)</code>	Synchronously call all listeners
<code>off(event, listener)</code>	Remove a specific listener
<code>removeAllListeners(event)</code>	Remove all listeners for event
<code>listenerCount(event)</code>	Number of listeners for event
<code>setMaxListeners(n)</code>	Increase listener limit (default: 10)

LESSON 5-6 • 13 min

Single-Threaded Node.js vs Multi-Threaded Servers

'Single-threaded' doesn't mean slow — it means the JavaScript execution runs on one thread, while I/O is handled by the OS and libuv's thread pool.

How Node.js Handles Concurrency

Node uses a single main thread for JavaScript execution. Blocking I/O is delegated to libuv, which uses a background thread pool (default size: 4). The main thread is freed immediately and processes other events while I/O completes.

```
// Non-blocking: main thread is NOT blocked
const fs = require('fs');
fs.readFile('large.csv', 'utf8', (err, data) => {
  // Runs AFTER file is read - back on main thread
  console.log('File size:', data.length);
});
console.log('This prints FIRST');
```

Worker Threads for CPU-Intensive Work

For CPU-bound tasks (image processing, cryptography, ML inference), use the `worker_threads` module to run code in parallel threads without blocking the event loop.

```
const { Worker, isMainThread, parentPort } = require('worker_threads');

if (isMainThread) {
```

```
const worker = new Worker(__filename);
worker.on('message', result => console.log('Result:', result));
} else {
  // CPU-heavy computation in worker thread
  const result = heavyComputation();
  parentPort.postMessage(result);
}
```

Performance Rule Never block the event loop with synchronous CPU work (e.g. large JSON.parse, crypto, image manipulation). Offload to Worker Threads or a separate microservice.

LESSON 5-7 • 6 min

How the Event Loop Works

The Event Loop is the heart of Node.js. It is a C program (inside libuv) that continuously checks whether there is work to be done and executes callbacks in a specific order across 6 phases.

Phase	Description
1. timers	Executes callbacks scheduled by <code>setTimeout()</code> and <code>setInterval()</code> whose delay has expired.
2. pending callbacks	Executes I/O callbacks deferred from the previous loop iteration.
3. idle / prepare	Internal use only (libuv housekeeping).
4. poll	Retrieves new I/O events and executes I/O callbacks. May block here waiting for events if the queue is empty.
5. check	Executes <code>setImmediate()</code> callbacks — always runs after the poll phase.
6. close callbacks	Handles 'close' events (e.g. <code>socket.on('close', ...)</code>).

Microtask Queue (Between Each Phase)

Between every phase transition, Node drains two microtask queues: `process.nextTick()` callbacks (highest priority) and resolved Promise callbacks.

```
// Execution order demonstration
setTimeout(() => console.log('1: setTimeout'), 0);
setImmediate(() => console.log('2: setImmediate'));
Promise.resolve().then(() => console.log('3: Promise'));
process.nextTick(() => console.log('4: nextTick'));
console.log('5: sync');

// Output: 5 → 4 → 3 → 1 → 2
```

LESSON 5-8 • 10 min

Streams, Buffers & Modular Systems Deep Dive

Streams are one of Node's most powerful features. Instead of loading entire files or payloads into memory, streams process data in chunks — enabling high-throughput, memory-efficient applications.

The 4 Stream Types

Type	Class	Use Case
Readable	<code>fs.createReadStream()</code>	Reading files, HTTP request body
Writable	<code>fs.createWriteStream()</code>	Writing files, HTTP response
Duplex	<code>net.Socket</code>	TCP sockets — read AND write
Transform	<code>zlib.createGzip()</code>	Modify data as it passes through

Piping Streams

```
// Compress a large file on-the-fly
const fs = require('fs');
const zlib = require('zlib');

fs.createReadStream('video.mp4')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('video.mp4.gz'))
  .on('finish', () => console.log('Done'));
```

Buffers

Buffers are fixed-size raw memory allocations outside the V8 heap, used to handle binary data (images, network packets, cryptographic keys).

```
const buf = Buffer.from('Hello, Node!');
console.log(buf); // <Buffer 48 65 6c ...>
console.log(buf.toString()); // 'Hello, Node!'
const safe = Buffer.alloc(256); // Zeroed buffer
```

LESSON 5-9 • 8 min

Harnessing IIFE as the Ultimate Module Wrapper

An IIFE (Immediately Invoked Function Expression) is a function that executes the moment it is defined. Node.js historically used IIFEs to implement its module system — every CommonJS module file is silently wrapped in one.

The CommonJS Module Wrapper

```
// Node wraps every .js file with this BEFORE execution:
(function(exports, require, module, __filename, __dirname) {
  // YOUR CODE IS INSERTED HERE
});
```

This is why `require`, `module`, `exports`, `__filename`, and `__dirname` appear to be global — they're actually injected as function parameters. The IIFE creates a private scope for every module, preventing variable leakage.

Why IIFEs Matter

- Each module gets its own scope — no global pollution
- `require`, `module`, `exports` are local, not truly global
- `__dirname` and `__filename` resolve to the actual file's directory/path
- Pattern still used in bundlers (Webpack, Rollup) for code isolation

IIFE Pattern in Application Code

```
// Useful for async top-level code (before ESM top-level await)
(async () => {
  const db = await connectDatabase();
  const app = express();
  app.listen(3000);
})();
```

LESSON 5-10 • 17 min

Exploring CommonJS & Name Alias

CommonJS (CJS) is the original Node.js module system. It uses synchronous `require()` calls and `module.exports` to define module boundaries. Despite ESM's emergence, CJS remains dominant in the Node.js ecosystem.

Exporting — 4 Patterns

```
// Pattern 1: Named exports
exports.add = (a, b) => a + b;
exports.PI = 3.14159;

// Pattern 2: Single default export
module.exports = class UserService { };

// Pattern 3: Export object literal
module.exports = { add, subtract, PI };

// Pattern 4: Factory function
module.exports = function createRouter(config) {
  return { get, post, put, del };
};
```


Importing with Name Aliases

```
// Destructure with alias (avoids name collisions)
const { add: sum, subtract: minus } = require('./math');
console.log(sum(2, 3)); // 5

// Classic alias pattern
const { createServer: createHttpServer } = require('http');
const { createServer: createHttpsServer } = require('https');
```

Module Caching

Node caches every required module after first load. Subsequent `require()` calls return the cached export — not a fresh copy. This is how singletons (DB connections, config) work in Node.

```
// Singleton pattern using CJS cache
// db.js
const connection = createDatabaseConnection();
module.exports = connection; // Same object everywhere it's required
```

LESSON 5-11 • 5 min

Transforming to ESM

ECMAScript Modules (ESM) is the official JavaScript module standard introduced in ES2015. Node.js gained stable ESM support in v12+. ESM uses static `import/export` syntax, enabling tree-shaking and top-level `await`.

Enabling ESM in Node.js

- Add `"type": "module"` to `package.json` (all `.js` files become ESM)
- Or rename files to `.mjs` extension
- CommonJS files must use `.cjs` extension in ESM projects

CJS vs ESM — Cheat Sheet

CommonJS	ESM Equivalent
<code>const x = require('mod')</code>	<code>import x from 'mod'</code>
<code>const { a } = require('mod')</code>	<code>import { a } from 'mod'</code>
<code>module.exports = x</code>	<code>export default x</code>
<code>exports.a = x</code>	<code>export const a = x</code>
<code>__dirname</code>	<code>import.meta.dirname</code>
<code>__filename</code>	<code>import.meta.filename</code>
<code>require()</code> anywhere	<code>import()</code> dynamic (async)

**Top-Level
Await (ESM
Only)**

In ESM modules you can use `await` at the top level without an `async` wrapper: `const data = await fetch(url).then(r => r.json());` — impossible in CommonJS.

APPENDIX

Core Modules Quick Reference

Node.js ships with a rich standard library. These are the modules every backend developer uses daily.

fs — File System

```
const fs = require('fs');
const fsp = require('fs').promises; // Promise-based

// Async read (promise)
const data = await fsp.readFile('file.txt', 'utf8');

// Write, mkdir, stat
await fsp.writeFile('out.json', JSON.stringify(obj, null, 2));
await fsp.mkdir('logs', { recursive: true });
const stat = await fsp.stat('file.txt'); // size, mtime...
```

http — HTTP Server & Client

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ status: 'ok' }));
});
server.listen(3000, () => console.log('Listening on :3000'));

// req.method, req.url, req.headers
```

path — File Path Utilities

```
const path = require('path');

path.join('/home', 'user', 'file.txt'); // '/home/user/file.txt'
path.resolve('src', 'index.js');        // absolute path
path.dirname('/home/user/file.txt');     // '/home/user'
path.basename('/home/user/file.txt');    // 'file.txt'
path.extname('file.txt');                 // '.txt'
```

os — Operating System Info

```
const os = require('os');

os.cpus(); // Array of CPU core info
os.freemem(); // Free memory in bytes
os.totalmem(); // Total memory in bytes
os.platform(); // 'linux', 'darwin', 'win32'
```

```
os.homedir(); // User home directory
os.hostname(); // Machine hostname
```

events — EventEmitter

```
const { EventEmitter, once } = require('events');

// See Lesson 5-5 for full API reference

// Promisify 'once' events
const [data] = await once(emitter, 'data');
```

Pro Tip: Build a small HTTP API using only Node's built-in modules (no Express, no frameworks). That hands-on exercise will solidify everything covered in this module.